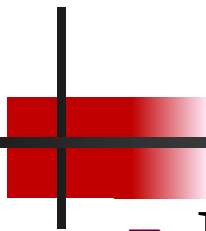


Network Security

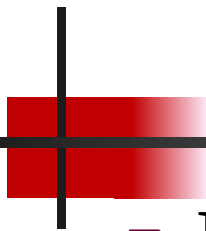
Overview

- ❑ Network security is the practice of **protecting computer networks and data** from unauthorized access, misuse, modification, or disruption.
- ❑ Core Objectives:
 - **Confidentiality:** Ensure that sensitive information is accessed only by **authorized users**.
 - **Integrity:** Safeguard data from unauthorized **modifications or corruption**.
 - **Availability:** Ensure network **resources are accessible** to legitimate users when needed.
 - **Authentication & Non-Repudiation:** **Verify user identities and ensure actions cannot be denied**.



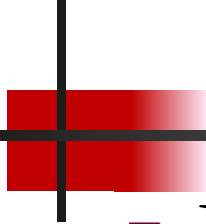
Key Components & Tools

- ❑ **Firewalls:** Control incoming and outgoing network traffic based on security rules.
- ❑ **Intrusion Detection/Prevention Systems (IDS/IPS):** Monitor / block suspicious activities.
- ❑ **Virtual Private Networks (VPN):** Provide secure remote access through encrypted tunnels.
- ❑ **Encryption Protocols:** Secure data transmission (e.g., SSL/TLS, SSH).
- ❑ **Network Segmentation:** Dividing the network to limit potential attack surfaces.



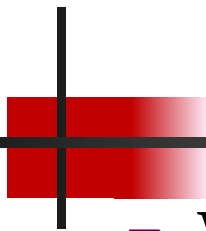
Common Threats

- ❑ **Malware & Ransomware:** Malicious software disrupting network operations.
- ❑ **Phishing & Social Engineering:** Techniques to trick users into revealing sensitive data.
- ❑ **Denial-of-Service (DoS) Attacks:** Overloading networks to cause outages.
- ❑ **Man-in-Middle (MiM) Attacks:** Intercepting communication between two parties.



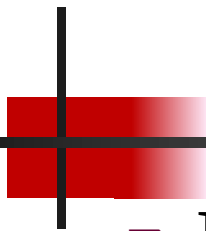
Best Practices

- ❑ Regular **patch management** and **software updates**.
- ❑ Implement **strong authentication** (e.g., multi-factor authentication).
- ❑ Continuous **monitoring** and **logging** of network activities.
- ❑ **User education** and **awareness** on security practices.



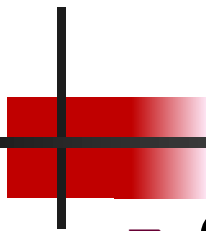
Software Vulnerabilities in Network Security

- ❑ What are Software Vulnerabilities?
 - **Flaws, weaknesses, or bugs** in software that can be exploited by attackers.
 - Can lead to **unauthorized access, data breaches, and system compromise.**
- ❑ Common Types of Software Vulnerabilities
 - Buffer Overflow
 - SQL Injection
 - Cross-Site Scripting (XSS)
 - Zero-Day Exploits
 - Weak Authentication & Password Management



Impact of Software Vulnerabilities

- ❑ **Data Breaches** – Leakage of sensitive personal or organizational data.
- ❑ **Denial-of-Service (DoS) Attacks** – Overloading a system to disrupt services.
- ❑ **Malware Injection** – Attackers plant viruses, ransomware, or spyware.



Buffer Overflow Vulnerability

- ❑ Occurs when a **program writes more data to a fixed-size memory buffer** than it can hold.
- ❑ **Overwrites adjacent memory, potentially corrupting data and control structures.**
- ❑ **How It Works:**
 - A buffer (e.g., an array in C/C++) is allocated a fixed size.
 - Excess input overwrites the buffer's boundaries.
 - Attackers can change return addresses or inject malicious code.



C++ Example

```
#include <iostream>
#include <cstring>
using namespace std;
int main() {
    char buffer[10]; // fixed-size buffer
    cout << "Enter your name: ";
    cin >> buffer;    // unsafe input (no bounds checking)
    cout << "Hello " << buffer << endl;
    return 0;
}
```

- Buffer can store only 9 characters + null terminator.
- If the user enters a longer string, memory beyond the array gets overwritten.

This may:

- corrupt nearby variables
- crash the program
- cause undefined behavior

Impact

❑ Impact & Examples:

- **Arbitrary Code Execution** (e.g., exploitation via unsafe functions like `gets()`).
- **System Crashes and Data Corruption** (e.g., early worm outbreaks like the Morris Worm-Released by Robert Tappan Morris in 1988, a graduate student at Cornell University. It became famous because it accidentally caused a large portion of the early internet to slow down or crash.).

❑ Mitigation Strategies:

- **Input Validation** and **bounds checking**.
- Use **secure functions** (e.g., `fgets()` instead of `gets()`).
- Implement **stack protections** (e.g., stack canaries-mechanism to detect many stack overflow attacks).

SQL Injection Vulnerability

- ❑ A security flaw that allows an attacker to **inject malicious SQL queries** through user inputs.
- ❑ How It Works:
 - **Unsanitized input** is directly included in SQL queries.
 - Attackers can **alter queries to retrieve, modify, or delete data**.
- ❑ Impact & Examples:
 - **Unauthorized Data Access:** Extraction of sensitive information
 - **Data Manipulation:** Insertion, updating, or deletion of database records.
 - **Common attack payload:** " ' OR '1'='1 " used to bypass authentication.

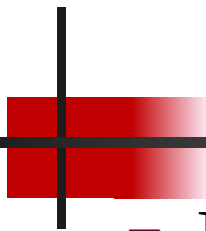
Example

Consider: Username: admin Password: 123

```
SELECT * FROM users WHERE username='admin' AND password='123';
```

- ❑ Injection Idea: If the program blindly trusts user input, the query logic can change. (SELECT * FROM users WHERE username = 'admin' OR TRUE;)
- ❑ "SELECT * FROM users WHERE username="" + username + "" AND password="" + password + """;

(Problem: User input is directly concatenated into the SQL query.)



Mitigation Strategies

- ❑ Use **Parameterized Queries** and **Prepared Statements**.
- ❑ **Validate and sanitize** all user inputs.
- ❑ Employ **ORM frameworks** that abstract direct SQL query formation.
- ❑ An **ORM (Object-Relational Mapping) framework**
 - maps database tables to programming objects, so developers can interact with databases using code, instead of SQL.

Cross-Site Scripting (XSS) Vulnerability

- ❑ A vulnerability where attackers **inject malicious scripts into web pages** viewed by other users.
- ❑ How It Works:
 - **Malicious JavaScript code is embedded into input fields.**
 - **The script is executed in the victim's browser**, compromising their session.
- ❑ Impact & Examples:
 - **Session Hijacking:** Stealing cookies or session tokens.
 - **Phishing and Redirection:** Redirecting users to malicious sites.
 - Types >> **Stored XSS** (script is permanently stored on the target server), **Reflected XSS** (script is delivered via a link), and **DOM-based XSS** (vulnerability exists entirely in the client-side JavaScript rather than the server-side code. The malicious script is executed when the browser manipulates the Document Object Model).

Example

- ❑ The page directly inserts user input into HTML.

```
<p>User comment: <span id="comment"></span></p>
```

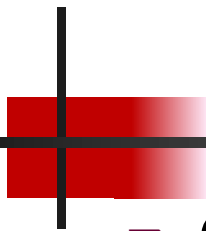
```
<script>
```

```
let userInput = "<user_input>";
```

```
document.getElementById("comment").innerHTML = userInput;
```

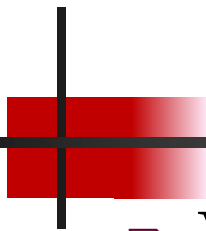
```
</script>
```

Imagine someone types Input that contains HTML with JavaScript inside it. The browser treats the input as **code**, not just text. So instead of just displaying it, the browser executes it.



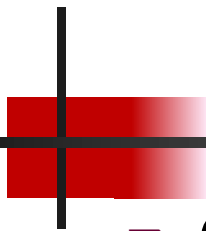
Mitigation Strategies

- ❑ **Output Encoding:** Ensure special characters are escaped.
- ❑ **Input Validation and Sanitization.**
- ❑ Implement a strong **Content Security Policy (CSP)**- it's a browser security mechanism that helps prevent attacks like XSS by controlling which content is allowed to run on a webpage.



Zero-Day Exploits

- ❑ Vulnerabilities **unknown to the software vendor and unpatched**, which attackers can exploit immediately.
- ❑ How It Works:
 - **Attackers find and exploit the flaw** before a patch is developed and distributed.
 - Can be used to deliver widespread attacks with little initial resistance.
- ❑ Impact & Examples:
 - **Severe Data Breaches:** Exploits may lead to significant data loss or system compromise. **Real-world examples include attacks like Stuxnet that leveraged multiple zero-day vulnerabilities. Discovered in 2010 that specifically targeted industrial control systems with nuclear facilities. It is widely considered one of the first cyber weapons designed to cause physical damage, not just steal data.**

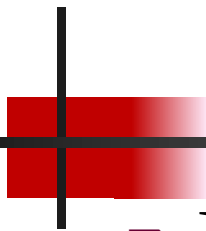


Mitigation Strategies

- ❑ **Continuous Monitoring and vulnerability scanning.**
- ❑ **Layered security measures** to limit damage.
- ❑ **Rapid patch management** and **incident response** once vulnerabilities are identified.

Weak Authentication & Password Management

- ❑ Inadequate methods for verifying user identity and managing passwords securely.
- ❑ How It Works:
 - Use of weak or default passwords, absence of multi-factor authentication (MFA), or insecure storage (e.g., plain text).
 - Attackers can exploit these weaknesses via brute-force attacks or credential stuffing.
- ❑ Impact & Examples:
 - **Unauthorized Access:** Compromise of user accounts and sensitive data.
 - **Credential Theft:** Facilitates identity theft and further intrusion into networks.



Mitigation Strategies

- ❑ Enforce **Strong Password Policies** (minimum length, complexity).
- ❑ Implement **Multi-Factor Authentication (MFA)**.
- ❑ Securely store credentials using techniques like **hashing** with salts (adding unique, random data-a "salt" to a password before it is hashed).